

Mandate — what it is, and how it meets every goal of this hackathon

Team 10 · Raingentic Commerce Hackathon NYC · 8–9 August 2026 Repo:

github.com/theomthakur/raingentic-team10

This is the plain-language version. No jargon, no assumed knowledge. If you read only one document about this project, read this one.

1. The problem, in ordinary words

AI agents are now good enough to do real work — find a supplier, compare prices, place an order. The moment they need to *pay*, everything stops, because paying means handing software a card number and hoping.

Rain solved a big part of this. Their **Agent Control Layer** lets you give an agent a card with rules attached: spend at most \$200, only at office-supply shops, only this month. Those rules are checked **before** the card is even created, not after the money is gone.

But there is a gap. Those rules can only describe **how much** and **where**.

An agent with a \$200 office-supply card can buy completely the wrong thing, at an allowed price, from an allowed shop, for a reason it made up. Every rule passes. The purchase is still wrong.

Nobody checks **why**.

2. What we built

Mandate makes the agent say why before it can pay, and then checks whether that's true.

Before any card exists, the agent has to hand over the actual purchase order it negotiated: which supplier, which item, what price, how many, valid until when. Plain code — not an AI — compares that against the company's real records.

If anything doesn't line up, **no card is ever created**.

That last part matters more than it sounds. This isn't a payment that gets declined. There is no card to decline, because the card was never made. Nothing to cancel, nothing to dispute, nothing to claw back.

The one-sentence version

Rain bounds how much an agent spends and where. Mandate checks why — and if the reason doesn't hold, the card is never issued.

If someone says "that's just a database check"

They're right, and that's the point. It's a **three-way match** — the same purchase-order / delivery / invoice cross-check every finance department on earth already does. We moved it from *after the invoice arrives* (where your only remedy is a dispute) to *before the card exists*.

3. How it works

Eight stages. Money only moves if stage 4 passes — and for large purchases, only once a person has also said so.

```
1 TASK      an agent is given a job to do
2 NEGOTIATE it collects quotes from competing suppliers and haggles one round
3 PROPOSE   it declares the winning quote as a purchase order
4 VERIFY    plain code checks that declaration against the real records
    |
    |— fails → REFUSE  no card, ever. A plain-English reason. Written down.
    |
    |— too big → HOLD  everything checked out, but it needs a person.
    |                  Still no card. A named human releases it, and
    |                  every check runs again on fresh records.
    |
    ▼
5 ISSUE     Rain creates a card scoped to exactly this purchase order
6 SETTLE    the purchase happens; the records are updated
7 REVOKE    the card is retired – it existed for exactly this purchase
8 RECORD    everything is filed permanently and can never be edited
```

There is no path in the code where a card gets made first and judged afterwards. The refusal branch simply never reaches Rain.

The seven checks

Each one returns a sentence a human can act on — never "validation failed."

#	The check	What it catches
1	Does this purchase order actually exist, and was it accepted?	An agent inventing a justification
2	Is it still open, and is the quote still valid?	Paying twice for something already delivered
3	Does the price match the quote?	Right supplier, wrong price
4	Does the supplier and the item match the quote?	Right supplier, right total, wrong item
5	Is there budget left?	Death by a thousand small, individually-fine purchases
6	Has a card already been issued for this order?	Paying twice because something was retried
7	Is it above the amount the agent may spend alone?	Large purchases going through with nobody looking

Check 4 is worth pausing on. A card company can limit the amount, the shop, the category, how often. It cannot tell you that you bought the *wrong item* from the *right shop* at the *right price* — because a card issuer has no idea your order system exists.

Check 6 is the one that carries the demo. More on that below.

Check 7 is the answer to the question everyone asks first. More on that immediately.

None of these rules were invented here

Every one implements a control that finance departments already run. Rules 1–4 are a **three-way match**, the purchase-order / delivery / invoice cross-check every ERP ships. Rule 6 is an **idempotency key**. Rule 7 is a **delegation of authority matrix**.

That matters more than it sounds. The honest answer to "*why should I trust software to spend my money?*" is not "our rules are clever." It is:

These are the controls your finance team already runs. We moved them to before the money is committed instead of after.

Each rule states its basis on screen and in the code. Sources in [CONTROLS.md](#).

3b. "But there's no human in the loop"

This is the first objection anyone raises, and it deserves a real answer rather than reassurance.

There is a human — above a limit you set

No company gives an employee unlimited spending authority either. It gives **bounded autonomy with an escalation path**: approve up to \$X alone, above that someone senior signs off. An agent gets exactly the same deal.

Above the configured limit (\$25,000 by default) a purchase is **held** — not refused. Every check passed; it is simply large enough that a person should look.

Three details that matter:

- 1. **No card exists while it waits.** Approving is what *creates* the instrument. A rejected escalation has nothing to cancel, dispute or claw back.
- 2. **The approver must be named.** An unattributed approval is not an approval — the whole point of the control is that someone accepted responsibility.
- 3. **Releasing re-runs every check** against fresh records. The world moves while things sit in queues; approval is permission to proceed, not a promise the facts still hold.

If two approvers both hit release, the second is refused — by rule 6, reading the record the first release wrote. Idempotency protecting a *human* race, not just a machine retry.

"Fine, but the AI could just decide it's allowed"

It cannot. **There is no AI anywhere in the decision path.** The agent proposes; ordinary, readable, testable code decides. Most "AI safety" claims ask you to trust a model's judgement about a model. This one does not ask that at all.

"Then couldn't you just change the rules?"

The sharpest version of the criticism, and the one that would make the audit worthless.

So changing policy takes **two people**. A rule change is written down as a *pending* version that decides nothing, and **the person who wrote it cannot be the person who activates it** — segregation of duties, the same control that stops whoever raises an invoice from also paying it. Every version is kept, hashed, and can be published to a public chain, so it also cannot be backdated.

Your worry	The answer
No human is involved	There is, above a limit you set
The AI decides what's allowed	It does not decide anything — it proposes
The rules could be wrong	They're the three-way match you already run
You could change the rules	Two people, and every version is kept and hashed
Prove it	Change a rule and re-run all history against it

The two ideas that make it more than a script

Rules are settings, not code. Each rule is a row in a table with a version number. Change one and it saves as a *new* version — the old one is never touched. A finance team edits policy without a developer.

The same inputs always give the same answer. There is no AI anywhere in the checking step. This isn't only about safety — it's what makes the next feature possible at all.

Replay — the feature nobody else will have

Because rules are versioned data, and because checking is perfectly repeatable, we can **re-run all of history against a rule you just changed**:

"Across 47 past decisions, 8 that we approved would now be refused. 0 refusals would now pass."

That's a real number from the running app, not a mock-up. It works because each decision saved a **photograph of the records as they looked at that moment**, rather than a link to records that have since changed. Replaying a link would judge today's facts and tell you nothing.

4. The demo

1. **Run a purchasing task.** Suppliers compete, one wins, a card is created for exactly that amount, expiring with the quote.
2. **Press the exact same button again.** Nothing new. No second scenario. No "bad agent" we wrote to fail. → **Refused.** Because the record the *first run itself wrote* now says this order is filled and already has a card. Rain is never contacted.
3. **Click the refusal.** Four fields: which rule failed, what it expected, what it got, and exactly which record it read. Anyone can audit it in five seconds.
4. **Run the capital purchase.** \$43,500 of conveyor line. Everything checks out — and it is **held**, not refused, because it is above what the agent may spend alone. No card exists. Type a name, release it, and *that* is what creates the card.
5. **Change a rule and hit replay.** *"Across 47 decisions, 8 approvals would now be refused."* Each row shows what the rule expected before and after, and why it flipped.
6. **Try to approve your own rule change.** Propose it under one name, then attempt to activate it under the same name. Refused. Ask someone else to type their name instead.
7. **Hand over the keyboard.** There's a form where anyone can write their own purchase order — change the supplier, the item, a single cent — and press issue. It goes through the identical code path. Nothing is special-cased for the demo.

Step 2 is the important one. Most demos show a failure the team wrote in advance. Ours is caused by the first half of our own demo. It can't be staged, because nothing was staged.

Steps 4 and 6 are the ones that answer *"why would I trust this?"* — and they answer it by being refused on stage, not by being asserted.

5. How this meets every goal of the hackathon

The event has three prize tracks plus one named theme. **One build qualifies for all of them** — not four projects.

Status is marked honestly:  **built and tested** ·  **built, waiting on a credential or confirmation** ·  **not built yet**

Track 1 — Best use of Rain

"Use Rain's payment infrastructure so an agent can transact by itself."

What we do	Status
The whole design copies Rain's own principle — enforce at issuance, not after the fact	
Cards are scoped to the exact purchase-order total, never a round number	
Card expiry is tied to the quote's expiry, so it can't outlive the job	
One card per order line, with the order line as the key, so retries can't double-spend	
Real calls to Rain's card-issuance API	 client written, endpoints need confirming on site
Retire the card once the job is done	 built — <code>PATCH /issuing/cards/{id}</code> confirmed; the status value it accepts is the last unknown

Why this is the strongest possible Rain submission: most teams will either rebuild Rain's control layer or ignore it. We extend it. Rain answers *how much* and *where*. We answer *why*, at the same moment, one level up. We're not competing with the Agent Control Layer — we're standing on it.

Being straight about it: cards are currently simulated. Everything around them is real — the checks, the scoping, the records, the refusals. The app labels every simulated card as simulated, on screen, so nothing is ever overstated. Swapping in the real call is one function, already written and wired: [lib/rain/issuer.ts](#).

Track 2 — General: "agents actually move money"

What we do	Status
An agent runs a real task end to end and produces a real purchase order	
Budgets are actually debited when a purchase is approved	
Every decision is filed permanently and can be audited	
A refusal produces no instrument at all — the strongest version of a control	
Real settlement across card rails	 same dependency as Track 1

The design survives the worst case. If sandbox payments don't work this weekend, most agent-commerce demos lose their story entirely. Ours doesn't, because our headline moment is a purchase that was **stopped**. Issuance is the only thing that has to work.

Track 3 — Agent negotiation

What we do	Status
Four supplier agents with genuinely different pricing strategies	✓
A buyer agent that makes a counter-offer; each supplier concedes differently	✓
Suppliers won't go below their floor, so the buyer doesn't always win	✓
The winner's quote becomes the purchase order everything else is checked against	✓
Optional AI-written dialogue from each supplier	✓ falls back silently if unavailable

The bit we're proud of: the negotiation isn't decoration sitting next to the real work. What it settles on gets **written into the records**, and the purchase order the agent then declares is checked against exactly that. Negotiation *causes* the thing that gets verified.

And we kept it honest. The prices are decided by fixed, inspectable rules, not by an AI improvising. Any AI involvement is cosmetic dialogue, clearly upstream, and never touches the checking step. A shallow fake negotiation would cost us credibility on the parts that are genuinely solid — and one of the judges does agent orchestration for a living.

Track 4 — Monad bounty

What we do	Status
Every rule version gets a fingerprint (SHA-256), stable and reproducible	✓
Storage for the on-chain transaction reference, and the UI badge that shows it	✓
The transaction itself — sends <code>version + fingerprint</code> to Monad testnet	✓ written and wired
Actually running it	🟡 needs a testnet RPC URL and a funded key

Why Monad genuinely matters here, rather than being decoration:

Replay proves our rules are data, not code. But it does *not* prove we didn't rewrite the rules afterwards to fit a history we already had. That's a real hole in an audit story, and "trust our timestamps" doesn't close it — the timestamps are ours too.

Publishing each rule version's fingerprint to a public chain closes it. Every decision that cites version 1 is then provably judged against rules that existed *before* it.

This answers Monad's own stated bar — "would it break at 15-second finality or 50 cents a transaction?" — honestly:

You have to anchor the rule versions, or the audit claim collapses. That's a handful of writes. But we also want to anchor every individual decision, and there are thousands. **On a chain costing 50 cents a write, you'd anchor the rules and give up on the decisions.** Only somewhere as cheap and fast as Monad can you afford both.

That's a real engineering trade-off, said plainly, rather than a token on-chain gesture.

How it's built. A zero-value transaction to our own address, carrying the version number and fingerprint as its payload — no smart contract to deploy and none to get wrong. Only *active* policy versions can be anchored: publishing a pending one would assert a policy exists when nobody has approved it.

What's left: a Monad testnet RPC URL and a funded testnet key. The code activates the moment those exist ([lib/monad/anchor.ts](#)). Without them the button is simply absent and everything else is unaffected — nothing in the decision path depends on the chain being reachable.

Where we stand overall

Track	Ready to demo	Gap
Best use of Rain	Yes	Card issuance path and auth confirmed against the live sandbox ; no collateral is linked yet, so cards have no spending power
Agents move money	Yes	Same
Agent negotiation	Yes, fully	None
Monad bounty	Yes, minus the send	A testnet RPC URL and a funded key

What we confirmed against Rain's live sandbox, rather than inferring from public docs — detail in [RAIN-API-CONFIRMED.md](#):

- The auth header is `api-key` , not an `Authorization` bearer. The API named the header in its own error message, so this is settled rather than guessed.
- The whole surface lives under `/issuing` . Card creation is `POST /issuing/users/{userId}/cards` .
- Our KYC is approved and the account is active.
- **No collateral contract is linked to the user**, so a card issued today has no spending power behind it. That is the open question for a Rain engineer, and it is the one thing standing between the current build and money genuinely moving.

One thing worth showing a judge: creating a card with Rain's *minimum* body gives you an active card, **no spending limit, and a six-year expiry**. That is the exact instrument Mandate exists to prevent — and the contrast with a card scoped to one purchase order and expiring with its quote makes the argument better

than any slide could.

6. What we chose not to build, and why

Saying no is a design decision too.

Idea	Why not
Budget hierarchies (parent agent splits budget among children)	Too close to what Rain's control layer already does. We'd be rebuilding their product in front of them.
Agent underwriting (a second pool takes on liability)	Needs Rain accounts we weren't given. Kept as one forward-looking sentence.
Live-streaming spend limits	Needs an unconfirmed Rain capability <i>and</i> a chain contract written from scratch. Two unknowns stacked on one afternoon.
AI agents voting on approval	Rejected on principle, not time. Putting an AI back into the checking step destroys the exact property that makes replay meaningful.

This restraint comes from losing a hackathon two months ago with roughly 85% of the winning system's substance. The winners' edge wasn't more features — it was editable, versioned rules and a re-run button. Replay is that lesson, built properly and from the start.

7. How it's built

Next.js 14, TypeScript, Tailwind. Deployed on Vercel — locally-hosted submissions are disqualified under the event's own rules.

Where	What
lib/checks/	The seven checks. No I/O, no AI, no clock-reading — the same input always gives the same answer
lib/rules/	Versioned rule data and the fingerprint that gets published to Monad
lib/replay/	Re-judging past decisions against a different rule version
lib/store/	The permanent record. Postgres when deployed, memory locally
lib/negotiation.ts	Competing suppliers, strategies, one counter-offer round
lib/monad/anchor.ts	Publishing a policy version's fingerprint to Monad testnet
lib/rain-client.ts	Everything that talks to Rain, in one place
lib/pipeline.ts	The seven steps, in order
app/api/purchase/	The full journey: negotiate → propose → verify → issue

41 automated tests cover the checks, the fingerprinting, replay, and the two human controls. They exist to prove the two claims the pitch rests on: that the same input always gives the same answer, and that every threshold lives in editable settings rather than in code.

One thing that must not be forgotten before submitting

The permanent record needs `DATABASE_URL` set on the deployed site. Without it, the log empties whenever the server goes idle — and replay, the feature we can't cut, quietly breaks on the exact link we submit. **It works perfectly on a laptop either way, which is what makes it dangerous.**

8. The 60-second pitch

- Rain gives an agent a card with a limit and an allowlist. That bounds **how much** and **where**.
- Nothing bounds **why**. An agent can buy the wrong thing at the right price from an allowed supplier, and every control passes.
- So we make the agent declare the purchase order it negotiated, and we check that against the real record. Six rules, all editable settings.
- If it doesn't hold, **no card is issued**. Not a decline — there's nothing to decline, because the card never existed.
- Which is Rain's own principle, enforcement at issuance, applied one level up.
- And because there's no AI in the checking step, we can re-run every past decision against a new rule and trust the difference.
- Above a limit you set, a **named person** releases it — bounded autonomy, the same deal a company gives an employee. No card exists while it waits, so approving is what creates the instrument rather than reviewing one already out in the world.
- And changing the rules takes **two people**, because whoever can raise a threshold could otherwise approve anything.